

PLAYER FSM 리팩토링 개선안

ProjectBase / Unity 6000.0.47f1 / 설계 문서 / 2026-09-16

상속 중심의 상태 구조를 행동과 전환 규칙으로 분리하는 제안입니다. Player, Monster, AIPlayer가 공통 상태를 재사용하면서 서로 다른 전환 규칙을 조립하도록 구성합니다.

문서 상태

대화에서 작성한 개선안과 예시 코드를 정리했습니다. 실제 게임 스크립트 수정은 적용되지 않았으며, 아래 코드는 설계 예시입니다. 전체를 그대로 복사하는 완성 패치가 아니며 Controller 통합과 애니메이션 원본 식별 구현이 추가로 필요합니다.

목표	설계 방향
상태 상속 축소	AgentStateBase 한 단계 아래에 Idle, Move, Jump, Fall, Attack, Hit, Death 배치
전환 판단 분리	ITransitionRule은 조건 검사만 수행. 목적지와 우선순위는 StateTransition에 보관
이벤트 책임 분리	입력은 요청 저장, 종료 이벤트는 완료 신호, 타격 이벤트는 전투 행동으로 처리
공격 비용 정확성	허용된 공격 전환이 최종 선택된 경우에만 타입 확정과 스테미나 차감
기존 동작 보존	점프 종료 조건과 Attack/Hit의 Idle 복귀는 첫 적용 단계에서 유지

범위 밖: 플랫폼 가장자리에서 밀려 올라오는 현상의 물리 수정. 콜라이더 형상, 수평 속도 적용, 접지 판정은 별도 재현과 검증이 필요합니다.

01. 구조와 이벤트 책임

현재 `GroundedIdleState`와 `GroundedMoveState`는 공통 행동을 확장하기보다 접지·점프 전환을 추가하기 위해 상속합니다. 이 조건을 외부 규칙으로 옮기면 `Player`와 `Monster`가 같은 `IdleState`와 `MoveState`를 사용할 수 있습니다.

입력 또는 이벤트	현재 역할	변경 후 위치
점프·공격 입력	현재 상태의 <code>OnInputEvent</code> 호출	<code>AgentSignals</code> 요청 저장 → 전환 규칙
공격·피격 애니메이션 종료	상태 내부 완료 플래그 변경	완료 신호 → 전환 규칙
공격 타격 프레임	<code>AttackState</code> 에서 피해 실행	전용 이벤트 수신부 → <code>CombatHandler</code>
사망 애니메이션 종료	사망 후처리 호출	<code>DeathState</code> 에서 완료 신호 확인 후 1회 처리

목표 데이터 흐름

입력 어댑터 → 요청 저장소 → 전환 조건 평가 → 우선순위 선택 → 확정 처리 → Exit / Enter

애니메이션 프록시 → 원본 상태·재생 검증 → 완료 신호 또는 타격 실행

Controller 상속은 첫 단계에서 유지

`AgentController` → `GroundedAgentController` → `PlayerController`는 즉시 해체하지 않습니다. 먼저 상태 계층과 전환 책임을 정리하고, 이후 필요에 따라 접지·점프 기능을 별도 컴포넌트로 분리합니다.

설계 원칙

조건 검사에는 스테미나 차감·요청 소비·공격 타입 변경을 넣지 않습니다. 상태 경과 시간은 FSM이 관리합니다. 사망 > 피격 > 행동 요청 > 일반 이동 순서로 우선순위를 정하고 한 번의 평가에서 최대 한 번만 전환합니다.

`AIPlayerInput`의 점프·공격은 값을 설정한 직후 초기화합니다. `Update`에서 현재 값만 읽는 방식으로 바꾸면 이 입력을 놓치므로 기존 이벤트 구독을 유지해야 합니다.

02. 적용 순서와 변경 목록

단계	작업	완료 기준
1	입력 요청·애니메이션 완료 저장소 도입	AI 순간 입력을 다음 평가까지 보존
2	전환 규칙과 FSM 평가 도입	우선순위, 상태 시간, 전환 1회 보장
3	Idle/Move 통합	Grounded 전용 파생 상태 제거
4	Jump/Fall 및 전투 상태 이전	상태에서 전환 판단 제거
5	이벤트·제네릭 기반 정리	상태에 행동 생명주기만 남김
6	Player/Monster/AIPlayer 검증	입력, 비용, 애니메이션 회귀 점검

상태 스크립트 변경

구분	대상	변경 내용
수정	IAgentState / AgentStateBase	OnInputEvent, OnAnimationEvent 제거. 단일 비제네릭 기반 유지
삭제 예정	GroundedAgentStateBase	Jump/Fall이 AgentStateBase 직접 상속
삭제 예정	GroundedIdleState / GroundedMoveState	공통 Idle/Move와 Player 규칙으로 대체
수정	IdleState / MoveState	입력과 전환 제거. 행동만 유지
수정	JumpState / FallState	점프·공중 이동 유지. 타이머와 전환은 FSM으로 이동
수정	AttackState / HitState	진입·종료 행동 유지. 완료 판정 외부화
수정	DeathState	완료 신호를 읽고 사망 후처리를 한 번만 실행
수정	AgentStateMachine	전환 평가, 우선순위, 시간, 상태 버전 관리
사용처 확인	StateMachine<T>	새 Agent FSM에서 사용하지 않음. 다른 사용처 확인 후 정리

03. Controller·지원 코드 변경 목록

구분	대상	변경 내용
추가	AgentSignals	점프·공격·피격·사망 요청과 완료 신호 저장
추가	TransitionContext	평가 시점의 DeltaTime과 상태 경과 시간
추가	ITransitionRule	부작용 없는 전환 조건 계약
추가	PredicateTransitionRule	간단한 조건을 함수로 주입
추가	GroundedFallTransition	접지 해제 조건 재사용
추가	StateTransition	출발·목적 상태, 우선순위, 조건, 확정 처리
수정	AgentController	피격·사망을 요청으로 저장. FSM 초기화·갱신 통합
수정	GroundedAgentController	점프 요청 저장. 접지 갱신 유지
수정	PlayerController	CanStartAttack 조회와 TryStartAttack 확정 분리
추가 구현	MonsterController	공격 타입 선설정 제거. Monster용 확정 처리
추가 구현	AgentAnimationEventProxy / IAgentAnimationListener	전용 수신 계약과 원본 재생 식별 연결
수정	PlayerStateFactory	상태와 전환 규칙을 함께 조립
추가 구현	MonsterStateFactory	점프·낙하 없는 공통 상태와 규칙 조립
유지	입력 Handler / AIPlayerInput	기존 이벤트 구독을 요청 저장소로 연결
유지	PlayerInputCommands.cs	자동 생성 코드와 키 바인딩 변경 없음

삭제 예정 항목은 설계상 통합 대상입니다. 실제 파일 삭제와 .meta 정리는 구현 시 사용처 및 직렬화 참조를 확인한 뒤 수행합니다.

04. 예시 코드: 요청과 전환 계약

```
public sealed class AgentSignals
{
    public int AttackRequest { get; private set; }
    public bool JumpRequested { get; private set; }
    public bool AnimationFinished { get; private set; }
    public int StateVersion { get; private set; }

    public void RequestAttack(int type) => AttackRequest = type;
    public void RequestJump() => JumpRequested = true;
    public void BeginState(int version)
    {
        StateVersion = version;
        AnimationFinished = false;
    }
    public void NotifyAnimationFinished(int sourceVersion)
    {
        if (sourceVersion == StateVersion)
            AnimationFinished = true;
    }
    public void ClearRequests()
    {
        AttackRequest = 0;
        JumpRequested = false;
    }
}
```

축약 예시입니다. 실제 저장소에는 HitRequested와 DeathRequested도 동일한 방식으로 추가하고 ClearRequests에서 초기화합니다.

```
public readonly struct TransitionContext
{
    public readonly float DeltaTime;
    public readonly float StateElapsedTime;
    public TransitionContext(float dt, float elapsed)
    {
        DeltaTime = dt;
        StateElapsedTime = elapsed;
    }
}

public interface ITransitionRule
{
    bool ShouldTransition(in TransitionContext context);
}
```

NextState를 규칙에 포함하는 원안도 가능합니다. 이 개선안은 조건 재사용과 구체 상태 클래스 의존 감소를 위해 목적지를 StateTransition.To(StateType)로 분리합니다.

05. 예시 코드: FSM 평가와 상태 행동

아래 Tick은 AgentStateMachine의 핵심 부분입니다. _transitions는 우선순위 내림차순으로 등록하며 같은 우선순위는 등록 순서를 유지합니다.

```
public void Tick(float deltaTime)
{
    if (CurrentState == null) return;
    StateElapsedTime += deltaTime;
    var context = new TransitionContext(deltaTime, StateElapsedTime);

    if (CurrentType != StateType.Death)
    {
        foreach (var t in _transitions)
        {
            if (t.From.HasValue && t.From.Value != CurrentType)
                continue;
            if (t.To == CurrentType) continue;
            if (!t.Rule.ShouldTransition(in context)) continue;
            if (t.TryCommit != null && !t.TryCommit()) continue;
            EnterState(t.To);
            break;
        }
    }
    _signals.ClearRequests();
    CurrentState.Execute();
}
```

EnterState는 이전 Exit → 현재 상태 교체 → 시간 초기화·버전 증가 → 신호 초기화 → 새 Enter 순서로 실행합니다. 목적 상태의 등록 여부는 전환 추가 시 검증합니다.

```
public abstract class AgentStateBase : IAgentState
{
    public virtual void Enter() { }
    public virtual void Execute() { }
    public virtual void FixedExecute() { }
    public virtual void Exit() { }
}

public sealed class MoveState : AgentStateBase
{
    private readonly System.Action<bool> _setMove;
    private readonly System.Action _move;
    public MoveState(System.Action<bool> setMove, System.Action move)
    {
        _setMove = setMove;
        _move = move;
    }
    public override void Enter() => _setMove(true);
    public override void FixedExecute() => _move();
    public override void Exit() => _setMove(false);
}
```

06. 예시 코드: 공격 확정과 애니메이션

입력 수신은 `Signals.RequestAttack`만 호출합니다. 아래 확정 함수는 허용된 상태의 Attack 규칙이 선택된 경우에만 실행합니다.

```
private int ResolveAttackType(int requested)
    => IsGrounded ? requested : 3;

public bool CanStartAttack(int requested)
{
    if (requested <= 0 || Stamina == null) return false;
    int type = ResolveAttackType(requested);
    if (type > _statData.attackDatas.Count) return false;
    if (_combatHandler.CurrentAttackType != 0) return false;
    float cost = _statData.attackDatas[type - 1].usedStamina;
    return cost >= 0f && Stamina.CurrentStamina.Value >= cost;
}

public bool TryStartAttack(int requested)
{
    if (!CanStartAttack(requested)) return false;
    int type = ResolveAttackType(requested);
    float cost = _statData.attackDatas[type - 1].usedStamina;
    if (!_combatHandler.SetAttackType(type)) return false;
    if (!Stamina.Use(cost))
    {
        _combatHandler.ResetAttackType();
        return false;
    }
    return true;
}

public void ReceiveAnimationEvent(AnimEventType type,
    StateType sourceState, int sourceVersion)
{
    if (_stateMachine.CurrentType != sourceState ||
        _stateMachine.StateVersion != sourceVersion) return;

    if (type == AnimEventType.OnFrame &&
        sourceState == StateType.Attack)
        _combatHandler.PerformAttack();

    if (type == AnimEventType.End)
        Signals.NotifyAnimationFinished(sourceVersion);
}
```

중요한 추가 구현: `sourceVersion`은 이벤트 도착 순간의 현재 버전이 아니라 재생 시작 때 캡처한 원본 버전이어야 합니다. Proxy와 Animator 상태/클립 이벤트의 연결, 블렌딩 중 이전 이벤트 배제는 별도 구현·검증해야 합니다.

07. 전환 구성 및 검증 계획

출발	조건	목적
사망 전 모든 상태	사망 요청 또는 체력 소진	Death
사망 전 상태	피격 요청	Hit
Idle / Move / Jump / Fall	유효한 공격 요청과 확정 성공	Attack
Idle / Move	접지 중 점프 요청	Jump
Idle / Move	접지 해제	Fall
Idle ↔ Move	이동 입력 유무	Move ↔ Idle
Jump	0.1초 이후 접지 / 비접지 + 애니메이션 종료	Idle / Fall
Fall	착지 후 이동 입력 유무	Move / Idle
Attack / Hit	해당 애니메이션 종료	Idle

Factory 조립 예시

```
states[StateType.Move] =
    new MoveState(player.Move, player.HandleMovement);

machine.AddTransition(new StateTransition(
    StateType.Move, StateType.Fall, 50,
    new GroundedFallTransition(() => player.IsGrounded)));

machine.AddTransition(new StateTransition(
    StateType.Move, StateType.Attack, 70,
    new PredicateTransitionRule(
        _ => player.CanStartAttack(signals.AttackRequest)),
    () => player.TryStartAttack(signals.AttackRequest)));
```

필수 검증

① AI가 즉시 초기화하는 입력도 1회 처리 ② Hit/Death에서 공격 비용 미차감 ③ 사망·피격·공격 동시 요청의 우선순위 ④ 평가당 전환 1회 ⑤ 재진입 시 타이머·완료 신호 초기화 ⑥ 이전 애니메이션 이벤트 무시 ⑦ 타격 프레임 정상 실행 ⑧ 사망 후처리 1회 ⑨ Player-Monster-AIPlayer 회귀 및 Unity 컴파일.

참조 범위

Assets/Scripts/FSM 하위 Agent/StateControl, Agent/Handler, Agent/@Hub, GroundedAgent, Player, NPC의 대화 중 확인한 소스와 리팩토링 예시를 기반으로 작성했습니다. 런타임 재현으로 플랫폼 문제의 원인을 확정한 문서는 아닙니다.